

Laboratory Data Upload & Integration Service

Message-driven file-processing microservice that ingests laboratory test data (Excel/CSV), validates and enriches records via SOAP/REST services, and updates the core inspection & measurement system — Java 11, Spring Boot 2.7, Spring Integration.

- Java 11
- Spring Boot 2.7
- Spring Integration
- Spring Integration SFTP
- Apache POI
- JAXWS / SOAP
- REST
- Oracle DB
- Hibernate Validator
- Jasypt
- OpenCSV
- Maven
- GitLab CI
- Ansible

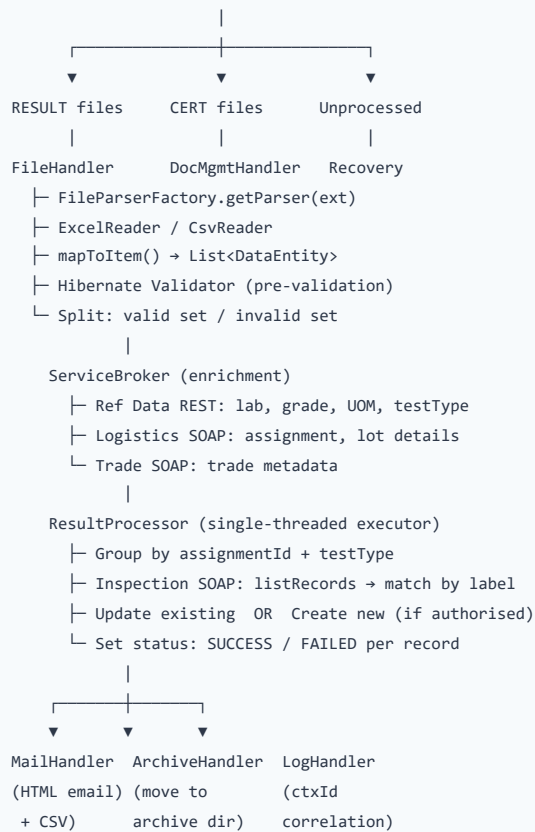
1 System Overview

A production-grade **file-driven integration service** that automates laboratory test data upload into a core trading platform's inspection & measurement system. An inbound file poller watches a local directory, routes files by type (result data vs. certificates), parses multi-format inputs (Excel, CSV), validates against business rules, enriches via reference data lookups, and persists to the inspection system via SOAP web services. Certificates are archived to a document management system.

Inbound Layer Spring Integration file poller (60s interval) Filename-pattern-based routing Factory-pattern file parsing (Excel/CSV) Hibernate Validator pre-validation Multi-format: XLSX, XLS, CSV, PDF stub	Processing Layer Spring Integration message channels Dedicated handlers per file type Reference data enrichment (labs, UOMs, grades) Assignment/lot matching via Logistics SOAP Single-threaded executor (sequential safety)	Outbound Layer Inspection SOAP service (create/update records) Logistics SOAP (assignment queries) Document management REST (certificate upload) Reference Data REST (lookups, caching) SMTP email notifications (HTML reports)
File-driven event architecture No REST API endpoint triggers processing — a Spring Integration poller watches a shared directory, enabling seamless integration with upstream file delivery systems (SFTP, email extractors).	Per-record error isolation Each row in an input file is processed independently. Failures on individual records do not reject the entire file — valid records proceed, failures are reported via email.	
Multi-service enrichment pipeline Raw file data is enriched via 5+ external service calls (reference data, logistics, trade) before inspection system update — ensuring data integrity at source.	Sequential processing for data consistency A single-threaded executor ensures record updates to the same assignment are never processed concurrently, preventing race conditions in the inspection system.	

2 File Processing Lifecycle

File Poller —scans directory (60s)→ Spring Integration Router



Why file-based, not API-driven? Laboratory systems and upstream extractors deliver data as files (via SFTP/email). The file poller pattern decouples the service from delivery mechanism specifics, and filename conventions encode routing metadata (lab name, assignment ID, file type) without requiring API contracts.

3 Message Handlers

Spring Integration routes files to dedicated handlers based on filename prefix. Each handler is a **ServiceActivator** bound to a named channel. A single-threaded `ResultExecutor` queues processing tasks sequentially to prevent concurrent updates.

File Handler (Result)

Channel: `resultChannel`

Parses RESULT/DATES files via factory-pattern readers (Excel, CSV). Validates each row with Hibernate Validator, enriches via reference data/Logistics lookups, then queues a `ResultProcessor` task for inspection system update.

Document Handler (Cert)

Channel: `certChannel`

Handles certificate PDF files. Authenticates with the document management system, creates a folder structure by assignment, uploads the file with metadata JSON, and sends a status email.

Unprocessed File Handler

Channel: `unprocessedChannel`

Recovery handler that scans the unprocessed directory for files that failed initial routing. Re-queues them for processing — prevents silent data loss from transient startup failures.

4 Data Design & Processing State Machine

Each row in an input file is modelled as a DataEntity with its own lifecycle state. Errors accumulate per entity — no batch rejection. Oracle views provide assignment and lot data for enrichment. Logistics DB accessed via JDBC for direct view queries.



After processing: valid + invalid results aggregated into HTML email report with CSV export. Source file moved to archive directory. Failed records include concatenated error details.

Data Source	Purpose	Key Fields
DataEntity (DTO)	Per-row domain model with 60+ fields and validation annotations	lineNumber, recordType, labName, assignmentID, testType, lotRef, result, UOM
Assignment View	Oracle view for logistics assignment data (direction, quality, product)	assignmentId, tradeDirection, quality, product, counterparty, quantities
Lot View	Oracle view for lot-level details within assignments	lotNumber, lotReference, dryQuantity, wetQuantity, analysisId
Reference Data Cache	In-memory cache of SRD lookups (labs, UOMs, commodities, elements)	refType, refCode, cachedValue
Archive Directory	Processed files moved here after completion	originalFileName, processedTimestamp

5 Key Architecture Patterns

Message-Driven File Processing (EIP)

Spring Integration poller + channel + handler pattern decouples file arrival from processing logic. Routing by filename prefix, not content inspection — fast and deterministic.

Factory Pattern File Parsing

FileParserFactory returns the correct reader (Excel, CSV, PDF) by file extension. Adding new formats requires only a new IFileParser implementation — no routing changes.

Per-Record Error Isolation

Each entity carries its own error state. Processing continues for valid records even when others fail — no batch rejection, graceful degradation with full error reporting via email.

Sequential Executor for Data Safety

ResultExecutor wraps a single-threaded ExecutorService. Tasks queued sequentially prevent concurrent inspection SOAP updates to the same assignment — eliminates race conditions.

Multi-Layer Validation Pipeline

Three validation stages: Hibernate annotations (format/nullability), business rules (SRD/assignment existence), processing rules (lot matching, UOM compatibility) — each stage filters before the next.

Resilient Retry with Backoff

All external service calls (SOAP + REST) implement retry logic — 5 attempts, 60s delay on HTTP 503 and socket timeouts. Exhaustion throws RuntimeException for error reporting.

Maven Profile-Based Multi-Environment

Environment-specific properties copied from profile folders (dev/qa/uat/prod) at build time. Jaspyt encrypts credentials at rest — master password provided at runtime via JVM arg.

Health-Check Gate Before Processing

ServerHealthCheckService pings all downstream endpoints (Inspection, Logistics, Reference Data) before processing begins — prevents wasted parsing and enrichment when services are unavailable.

6 Architecture Decisions

Decision	Choice	Rationale
Integration style	Spring Integration (file poller + channels)	File-based input requires EIP patterns; Spring Integration provides declarative polling, routing, and handler binding without custom scheduler code
File parsing	Apache POI + OpenCSV + Factory pattern	Labs deliver Excel and CSV; POI handles formula evaluation; factory pattern makes format addition trivial
External services	SOAP (Inspection, Logistics, Trade) + REST (Ref Data, DMS)	Inspection and Logistics expose WSDL-only interfaces; reference data and document management offer REST; both protocols supported natively
Concurrency model	Single-threaded executor	Inspection SOAP service is not thread-safe for concurrent updates to the same assignment; sequential processing eliminates data corruption risk
Error strategy	Per-record accumulation, no batch rejection	A single bad row should not block 100 valid rows; errors concatenated per entity and reported via email
Credential security	Jasypt encryption + JVM arg master password	Encrypted at rest in properties files; master password never committed to source control
Deployment package	Assembly ZIP (JAR + libs + WSDL + shell script)	Target hosts require full distribution; assembly plugin creates a self-contained deployable without Docker dependency

7 CI/CD Pipeline



Semgrep runs as a SAST security scanner on every commit. Assembly ZIP deployed via Ansible playbooks: **stop** → **deploy** → **enable service** → **start** → **health check**. Four environment profiles: DEV, QA, UAT, PROD.

8 Observability

Correlation Logging

Each file generates a unique ctxId hash propagated through all Spring Integration channels and handlers. Log entries traceable end-to-end per file.

Health Check Gate

ServerHealthCheckService pings Inspection, Logistics, and Reference Data endpoints before processing. Prevents wasted work when downstream services are unavailable.

Email Notifications

HTML status reports sent per file: line-by-line results table, error details, and CSV export. Separate emails for validation failures and certificate upload status.

File Archival

Every processed file moved to a timestamped archive directory. Provides audit trail and enables manual reprocessing if needed.

Structured Logging

Logback-based logging with per-environment configuration. On-host logs written to application log file; container deployments forward to centralized log aggregation.

Error Accumulation

Per-entity error fields concatenate all failure reasons across validation, enrichment, and processing stages — full diagnostic context in a single email row.

9 Technology Stack

Java 11 Spring Boot 2.7.12 Spring Integration Spring Integration SFTP Apache POI 4.1 OpenCSV 5.5 JAXWS 2.6 (SOAP) RestTemplate (REST) Oracle DB (ojdbc8) Hibernate Validator Jasypt Joda-Time 2.12 JavaMail 1.4 SAAJ (SOAP Attachments) Logback JUnit Maven 3.8+ Maven Assembly Plugin jaxws-maven-plugin GitLab CI/CD Semgrep SAST Ansible